



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2233 — PROGRAMACIÓN AVANZADA 2026-2

Forma A

Interrogación 1

10 de Septiembre 2026

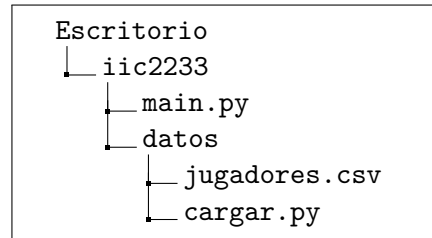
Instrucciones

- La evaluación consta de 3 preguntas de desarrollo.
- Debes completar con tus datos personales e indicar la forma en cada hoja de respuesta utilizada. Luego debes entregar de vuelta **todas** las hojas de respuesta de la prueba, aunque estas se encuentren en blanco.
- Solo podrás retirarte de la sala una vez hayan transcurrido 30 minutos desde que inició de la evaluación.
- Durante la evaluación se realizarán 2 rondas de preguntas. Estas preguntas únicamente serán respondidas por los profesores del ramo.
- En caso de que sea necesario, podrás solicitar hojas blancas para apoyar al desarrollo de la prueba. Para esto levanta la mano y espera que un ayudante se acerque a tu puesto.

Pregunta 1. [30 % nota final]

Parte A. (2 puntos)

Ana tiene la siguiente estructura dentro de su Escritorio:



El archivo `cargar.py` contiene la línea:

```
open("jugadores.csv")
```

Con la terminal ubicada en `iic2233`, Ana ejecuta:

```
python datos/cargar.py
```

y obtiene el siguiente error:

```
FileNotFoundError: [Errno 2] No such file or directory: 'jugadores.csv'
```

- A.1) Explica en máximo 2 líneas por qué ocurre el error, si el archivo `jugadores.csv` sí existe.
- A.2) Escribe una secuencia de comandos que permita a Ana ejecutar correctamente `cargar.py`, sin que se produzca el error anterior.

Parte B. (2 puntos)

Un ayudante debía procesar un archivo de registros de **1 GB**. Para hacerlo, escribió dos versiones de un programa: en la primera utilizó `archivo.readlines()` y recorrió la lista resultante; en la segunda, recorrió el archivo mediante un generador que entregaba una línea a la vez. Ambas versiones entregaron exactamente el mismo resultado, pero la segunda utilizó mucha menos memoria.

- B.1) Explica en máximo 3 líneas por qué la segunda versión utilizó menos memoria, si de todas formas ambas terminaron recorriendo todas las líneas del archivo.
- B.2) Su compañera quiere aplicar la misma idea en otro programa. En este caso, el programa debe leer todas las líneas del archivo y luego imprimirlas ordenadas alfabéticamente según el contenido completo de cada línea. ¿Se mantiene la ventaja de memoria en ese caso? Justifica en máximo 3 líneas.

Parte C. (2 puntos)

Considera las siguientes situaciones relacionadas con estructuras de datos en Python.

C.1) Se ejecuta el siguiente código:

```
from collections import namedtuple

Cancion = namedtuple("Cancion", ["titulo", "artista", "duracion"])

cancion_1 = Cancion(
    titulo="Dreams",
    artista="Fleetwood Mac",
    duracion=257
)
```

Luego se intenta ejecutar:

```
cancion_1.duracion = 300
```

Explica en máximo 2 líneas si esta expresión funciona correctamente o genera un error, y por qué.

C.2) En otro programa de Python que gestiona un curso, se definen dos estructuras de datos de tipo `set`: `inscritos`, que almacena los nombres de quienes se inscribieron al curso, y `asistentes`, que almacena los nombres de quienes fueron a la primera clase.

Escribe una expresión que entregue a las personas inscritas que **no asistieron** a la primera clase.

C.3) En el mismo programa del inciso (C.2) se intenta obtener un elemento de `inscritos` mediante la siguiente expresión:

```
inscritos[0]
```

Explica en máximo 2 líneas si esta expresión funciona correctamente o genera un error, y por qué.

Pregunta 2. [30 % nota final]

El Bingo es un clásico juego de azar en el cual cada jugador tiene un cartón con 15 números del 1 al 100. El director del juego saca números aleatorios de una tómbola y anuncia el número en voz alta. Cada jugador que tenga el número anunciado en su cartón lo marca con un plumón. Cuando alguien completa todo su cartón, grita bingo y gana un premio.

Para crear los cartones de bingo, alguien hizo la siguiente clase:

```
class CreadorCartones:
    def __init__(self, total_cartones: int):
        self.cartones = []
        self.total_cartones = total_cartones
        self.generar_cartones()

    def generar_cartones(self) -> None:
        for carton in range(self.total_cartones):
            numeros_random = generar_numeros()
            self.cartones.append(numeros_random)
```

La función `generar_numeros` retorna una lista ordenada con 15 números distintos del 1 al 100.

El problema de `Generador` es que hay veces en que se generan cartones repetidos, lo cual no debería pasar en el bingo.

- A) (4 puntos) Usando las estructuras de datos aprendidas en el curso, reescribe la clase `Generador` para que se asegure de que no se generen cartones repetidos. Utiliza pseudocódigo.

Tu programa debe usar las estructuras de datos de manera eficiente.

Puedes usar `generar_numeros` en tu código.

- B) (2 puntos) Explica las propiedades de la o las estructuras de datos usadas que permiten que el nuevo código sea más eficiente.

Pregunta 3. Sistema TAG [40 % nota final]

Una concesionaria utiliza un sistema TAG para calcular y registrar los pagos de los vehículos que atraviesan sus pódicos. Cada vehículo posee una patente y paga un monto calculado a partir de la tarifa base del pódico. Los automóviles pagan la tarifa base que es una constante del sistema, los camiones pagan el doble y las motocicletas pagan la mitad. Para simplificar, todas las tarifas base serán números enteros positivos y pares, por lo que los montos pagados también serán números enteros positivos.

Existe un módulo que contiene las clases `SistemaTAG` y `RegistroPago`. Ambas ya se encuentran implementadas y no deben modificarse. Cada objeto `RegistroPago` posee los atributos `patente: str` y `monto: int`.

Puedes utilizar las siguientes operaciones de `SistemaTAG`:

- `registrar_vehiculo:`

Recibe un vehículo y lo incorpora al sistema. No retorna ningún valor.

- `buscar_vehiculo:`

Recibe una patente y retorna el vehículo asociado. Si la patente no se encuentra registrada, levanta `KeyError`.

- `registrar_pago:`

Recibe un vehículo y la tarifa base del pódico. Calcula el monto mediante el método `calcular_peaje` del vehículo recibido. Si la tarifa base es menor o igual a cero, levanta `ValueError` y no registra el pago. En caso contrario, conserva la patente y el monto pagado en un nuevo objeto `RegistroPago`, lo registra y lo retorna.

- `obtener_registros:`

Retorna una lista de objetos `RegistroPago` en el mismo orden en que fueron agregados mediante `registrar_pago`. El primer elemento corresponde al primer pago registrado, el segundo al segundo pago registrado, y así sucesivamente. Los registros no se ordenan por patente ni por monto.

Un mismo vehículo puede realizar varios pagos. Todos estos pagos deben conservarse, incluidos aquellos asociados a una misma patente, y mantenerse en el orden en que fueron registrados.

Los pódicos comunican cada cobro mediante un texto con el formato `PATENTE;TARIFA_BASE`. Por ejemplo, `ABCD12;1500` representa un cruce de la patente `ABCD12` con tarifa base 1500. Una lectura es inválida si no contiene exactamente una patente no vacía y una tarifa base que pueda convertirse a un entero positivo. Estos casos deben manejarse mediante `ValueError`. Si la patente no está registrada, debe manejarse el `KeyError` levantado por `buscar_vehiculo`. Ante cualquiera de estos errores, el programa debe mostrar un mensaje en consola, no registrar el pago y continuar funcionando.

Consideraciones

- En el manejo de errores se deben distinguir las situaciones asociadas a `ValueError` y `KeyError`. Una solución que capture únicamente `Exception`, sin considerar las excepciones específicas descritas en el enunciado, no se considerará correcta.

A partir de lo anterior, responde las siguientes preguntas:

- A) **(2 puntos)** Construye un diagrama de clases UML para modelar el problema. El diagrama debe incluir **SistemaTAG**, **RegistroPago**, y todas las otras clases necesarias para el correcto modelamiento del problema.

Justifica brevemente cómo tu modelación permite que el sistema registre el pago de cualquier tipo de vehículo.

- B) **(2.5 puntos)** Escribe en pseudocódigo la siguiente función:

```
procesar_lectura(sistema: SistemaTAG, lectura: str)
```

La función debe revisar la lectura y buscar el vehículo correspondiente. Si el pago puede registrarse, debe retornar el objeto **RegistroPago** creado. Si ocurre alguno de los errores descritos en el contexto, debe mostrar el mensaje correspondiente en consola, continuar la ejecución y retornar **None**.

- C) **(1.5 puntos)** Escribe en pseudocódigo la siguiente función:

```
consultar_patentes(sistema: SistemaTAG, monto_minimo: int)
```

La consulta debe entregar, una por una, las patentes asociadas a pagos mayores o iguales al monto mínimo, respetando el orden de los registros y sin almacenar previamente todos los resultados. Si una patente aparece en más de un registro que cumple la condición, debe entregarse una vez por cada uno de esos registros.

Para probarla, considera los siguientes registros.

Patente	Monto pagado
AA1111	1500
BB2222	2000
AA1111	2001
CC3333	2500
AA1111	2400

Si `monto_minimo` es 2000, la consulta debe entregar, en este orden, BB2222, AA1111, CC3333 y AA1111.

Nombre completo: _____

RUN: _____ Sección: ____ Forma: ____ N° lista: _____ Página: ____ de ____

IIC2233 - Pregunta 1

Nombre completo: _____

RUN: _____ Sección: ____ Forma: ____ N° lista: _____ Página: ____ de ____

IIC2233 - Pregunta 2

Nombre completo: _____

RUN: _____ Sección: ____ Forma: ____ N° lista: _____ Página: ____ de ____

IIC2233 - Pregunta 3